

QEMU - Virtualisierung auf der Kommandozeile

Thomas Huth
@ Tübix 2026



Rechtliches

- Lizenz des “Jungle Leaves” Hintergrunds:
CC BY 3.0 US
<https://creativecommons.org/licenses/by/3.0/us/>
Bild angepasst vom Original, das hier zu finden ist:
<https://www.freevector.com/jungle-leaves-vector-background>
- Lizenz des “Kew the Angry Emu” Bilds von Benoît Canet:
CC BY 3.0
<https://creativecommons.org/licenses/by/3.0/>
Bild angepasst vom Original, das hier zu finden ist:
https://gitlab.com/qemu-project/qemu/raw/v8.0.0/pc-bios/qemu_logo.svg

Was ist QEMU?

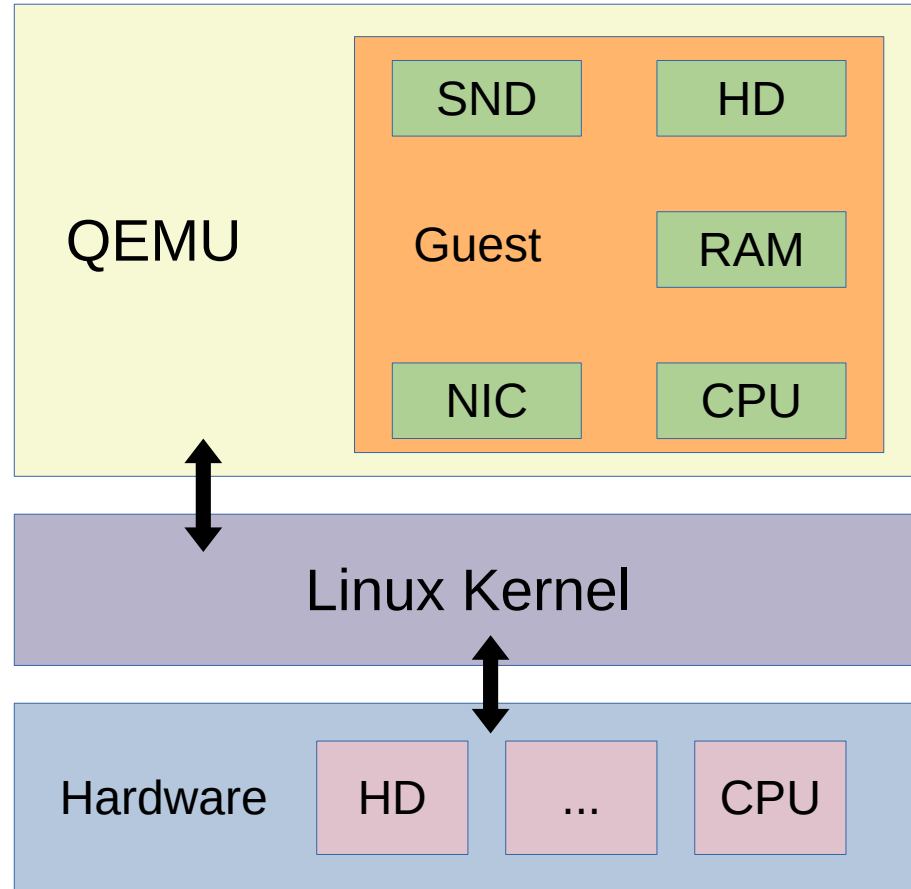
- Quick **EMU**lator – <https://www.qemu.org/>
- „A generic and open source machine emulator and virtualizer“
- Generic - Nicht auf eine Ziel-CPU festgelegt:
alpha, arm, avr, hexagon, hppa, loongarch,
m68k, microblaze, mips, or1k, ppc, riscv, rx,
s390x, sh4, sparc, tricore, x86, xtensa
- Open Source: GPLv2

Emulator & Virtualizer?

- Unterschiedliche Modi:
- „user mode“ Emulation zum Ausführen von Linux-Binaries (qemu-arm, ...)
- „system mode“ Emulation zum Ausführen ganzer Betriebssysteme (qemu-system-arm, ...)
 - Vollständige Emulation (TCG)
 - Virtualisierung (KVM)

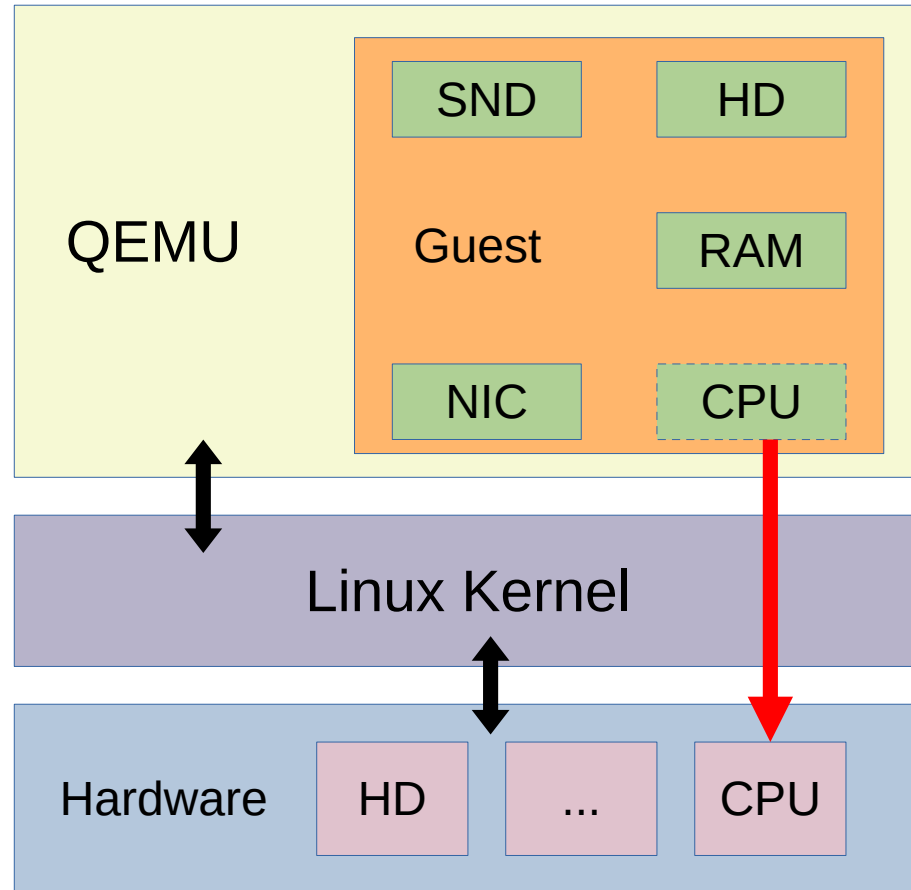
System Emulation (TCG)

- Bildet ganzen Computer nach
- TCG = Tiny Code Generator (JIT CPU emu)
- Alle anderen CPU Architekturen möglich!

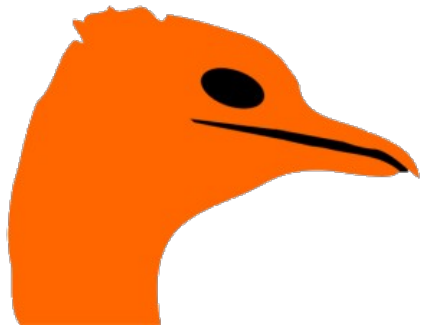


System Emulation (KVM)

- CPU wird via Host-CPU virtualisiert
- KVM = Kernel Virtual Machine („Accelerator“)
- Nur CPU Architektur wie Host möglich!



QEMU auf der Kommandozeile



Kommandozeilen-Parameter

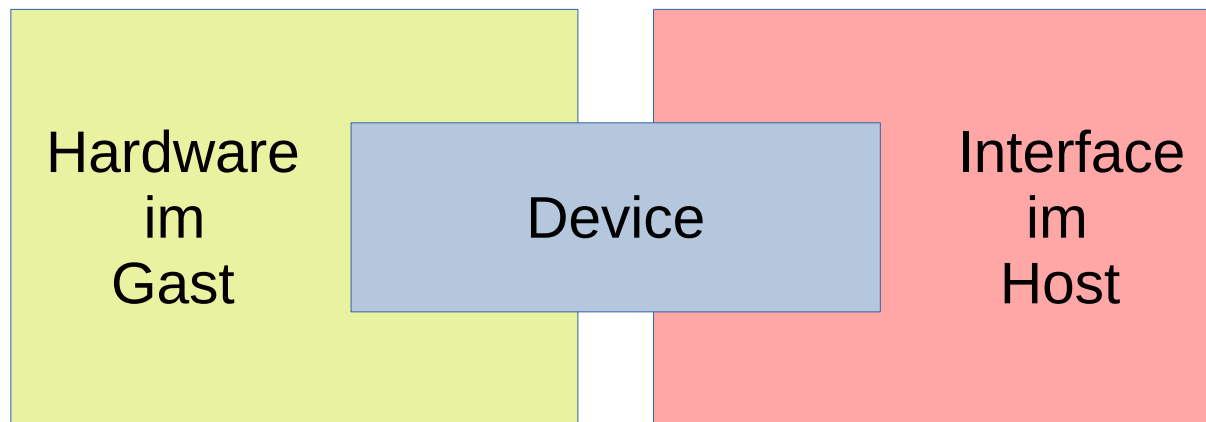
- Kein Unterschied zwischen „-“ und „--“ :
`-h = --h = -help = --help`
- Hilfe bei vielen Parametern mit „help“,
z.B.:
 - `-machine help` (zeige verfügbare Maschinen)
 - `-device help` (zeige verfügbare Geräte)

Hilfe bei „-device“

- `$ qemu-system-x86_64 -device e1000,help`
[...]
`e1000.addr=int32` (PCI slot and function number)
`e1000.x-pcie-extcap-init=bool` (on/off)
`e1000.extra_mac_registers=bool` (on/off)
`e1000.mac=str` (Ethernet 6-byte MAC Address...)
`e1000.netdev=str` (ID of a netdev backend)
- `$ qemu-system-x86_64 -device \`
`e1000,mac=52:54:00:12:34:56,addr=06.0`

Zweiteilung von I/O-Geräten

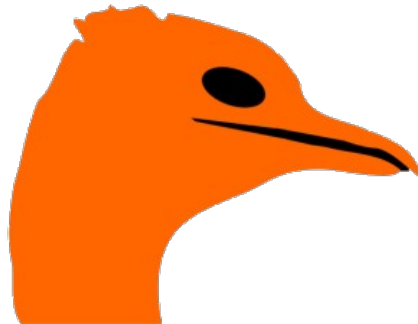
- Emulierte Hardware auf Gast-Seite (z.B. e1000 Netzwerkkarte)
- „Backend“ auf Host-Seite (z.B. „tap“ Netzwerkschnittstelle)



Wissenswertes zu CLI-Parameter

- Vieles historisch gewachsen (23 Jahre...)
 - „Legacy“ Parameter (altes Zeug!)
z.B.: `-net` oder `-drive`
 - „Architected“ Parameter (für vollen Zugriff)
z.B.: `-netdev`, `-blockdev` oder `-device`
 - „Convenience“ Parameter (für Benutzer*)
z.B.: `-nic` oder `-hda`

Zeichenbasierte Geräte



Character devices

- Verfügbare Backends anzeigen:
-chardev help
- Gibt z.B.: file, socket, stdio, pipe, tty, ...
- Ausgabe einer seriellen Schnittstelle in Datei:

```
$ qemu-system-x86_64 \
    -chardev file,id=c1,path=io.txt \
    -device isa-serial,chardev=c1
```

Chardevs – Alternative Parameter

- Legacy / convenience Schnittstellen-Parameter:

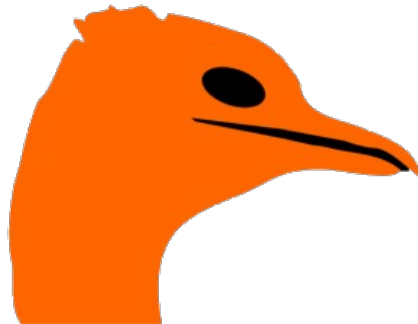
`-serial` und `-parallel`

- Nützlich bei Maschinen ohne Grafikkarte:

```
$ qemu-system-ppc -M ppce500 \  
    -display none \  
    -serial mon:stdio
```

- `mon:stdio` stellt sowohl den QEMU-Monitor als auch die serielle Ausgabe des Gastes auf `stdio` bereit (umschaltbar mit CTRL-a c)

Netzwerkgeräte



Netzwerkgeräte

- **Verfügbare Netzwerkbackends:**

- netdev help

- user oder passt : NAT, ohne Sonderrechte

- tap : „echtes“ Netzwerk über Linux TAP

- socket : Pakete über einen Socket tunneln

- **Beispiel:**

- ```
$ qemu-system-x86_64 \
```

- ```
-device virtio-net,netdev=n1 \
```

- ```
-netdev user,id=n1,dhcpstart=10.0.0.50
```

# Netzwerk – Alternative Parameter

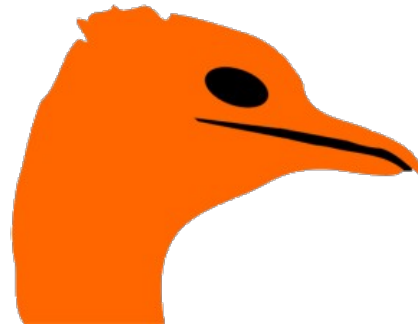
- „Convenience“ Parameter für einfache Fälle:  
-`nic` kann beides gleichzeitig!

- Beispiel:

```
qemu-system-x86_64 \
 -nic user,model=igb,tftp=/tmp/tftp
```

- Legacy-Parameter „-net“:  
Bitte nicht mehr benutzen!

# Blockgeräte



# Block devices

- Block-Backends werden „gestapelt“ (Zugriffsart, Dateiformat, ...), z.B.:

```
qemu-system-x86_64 -m 1G \
-blockdev file,node-name=f1,filename=img.qcow2 \
-blockdev qcow2,node-name=q1,file=f1 \
-blockdev blkdebug,node-name=d1,config=cnf.txt,image=q1 \
-device ide-hd,drive=d1
```

- Protokolle: file, https, nbd, ssh, ...
- Formate: qcow2, vhdx, vdi, vmdk, ...

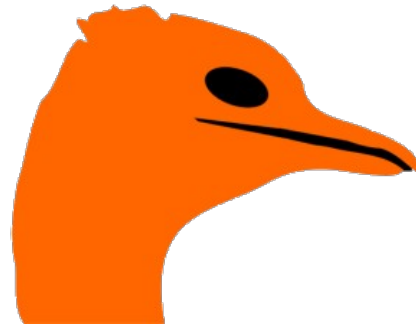
# Block Legacy/Convenience

- Mit dem alten -drive Parameter geht beides zusammen:

```
$ qemu-system-x86_64 \
 -drive if=ide,format=qcow2,file=disk.qcow2
```

- Mit „if=none“ kann man auch nur Backend konfigurieren
- Weitere Convenience Parameter:  
-fda , -cdrom , -pflash , -hda , ...

# Weitere wichtige Parameter



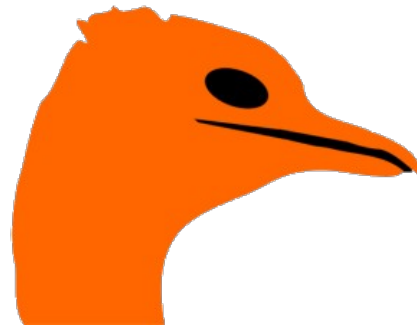
# CPU + RAM Parameter

- `-accel kvm` : KVM statt TCG benutzen
- `-cpu help` : Zeigt mögliche Gast-CPUs an
  - `-cpu host` : Gleiche CPU im Gast wie Host
  - `-cpu max` : Bestmögliche CPU unter TCG
- `-smp <n>` : Für mehrere CPUs im Gast
- `-m <size>` : Speicher (RAM) für den Gast

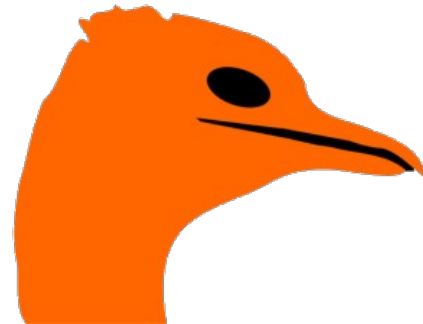
# User Interface

- `-nographic` : Keine GUI, nur Console
- `-display help` : Verfügbare Uls anzeigen
- `-display gtk` : GTK interface ( → Menüs)
- `-display sdl` : libSDL interface
- `-vnc :1` : VNC server starten

Demo



Der größere Zusammenhang  
—  
Oder:  
Warum eigentlich so kompliziert?

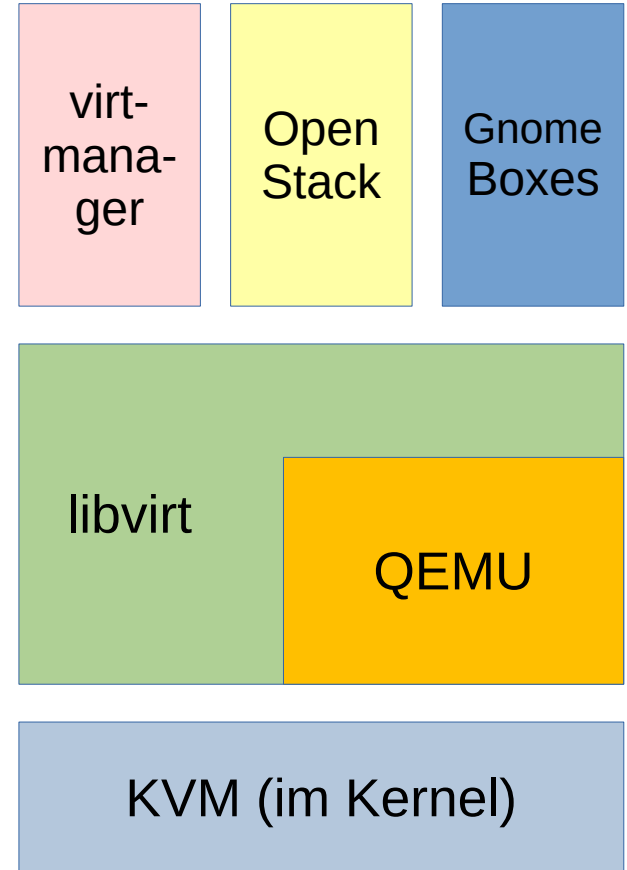


# Warum so kompliziert?

- QEMU versteht sich als „Building-Block“ (z.B. „-blockdev“ ist eher für andere Software und nicht für Menschen gedacht)
- Themen, um die sich QEMU *nicht* selbst (komplett) kümmert:
  - Einfache graphische Oberfläche
  - Life-Cycle-Management von Maschinen
  - Security (z.B. SELinux)
  - ...

# Der OSS Virtualisierungs-Stack

- Libvirt:
  - Abstraktion vom VMM
  - Life-Cycle Management
  - API
  - SELinux
  - Migrations-Management
  - ...
- Upper Layer:  
virt-manager, Gnome Boxes,  
Open Stack, oVirt, Kimchi, Cockpit



# libvirt

- Definition von Maschinen über XML (erstellbar z.B. mit „virt-install“):

```
<domain type='qemu'>
 <name>MeineVM1</name>
 <memory>4096</memory>
 <vcpu>2</vcpu>
 <os><type arch='x86_64' machine='q35'>hvm</type></os>
 <devices>
 <emulator>/usr/bin/qemu-system-x86_64</emulator>
 <!-- ... -->
 <interface type='network'>
 <source network='default' />
 </interface>
 <graphics type='vnc' port='-1' />
 </devices>
</domain>
```

# libvirt virsh Kommandos

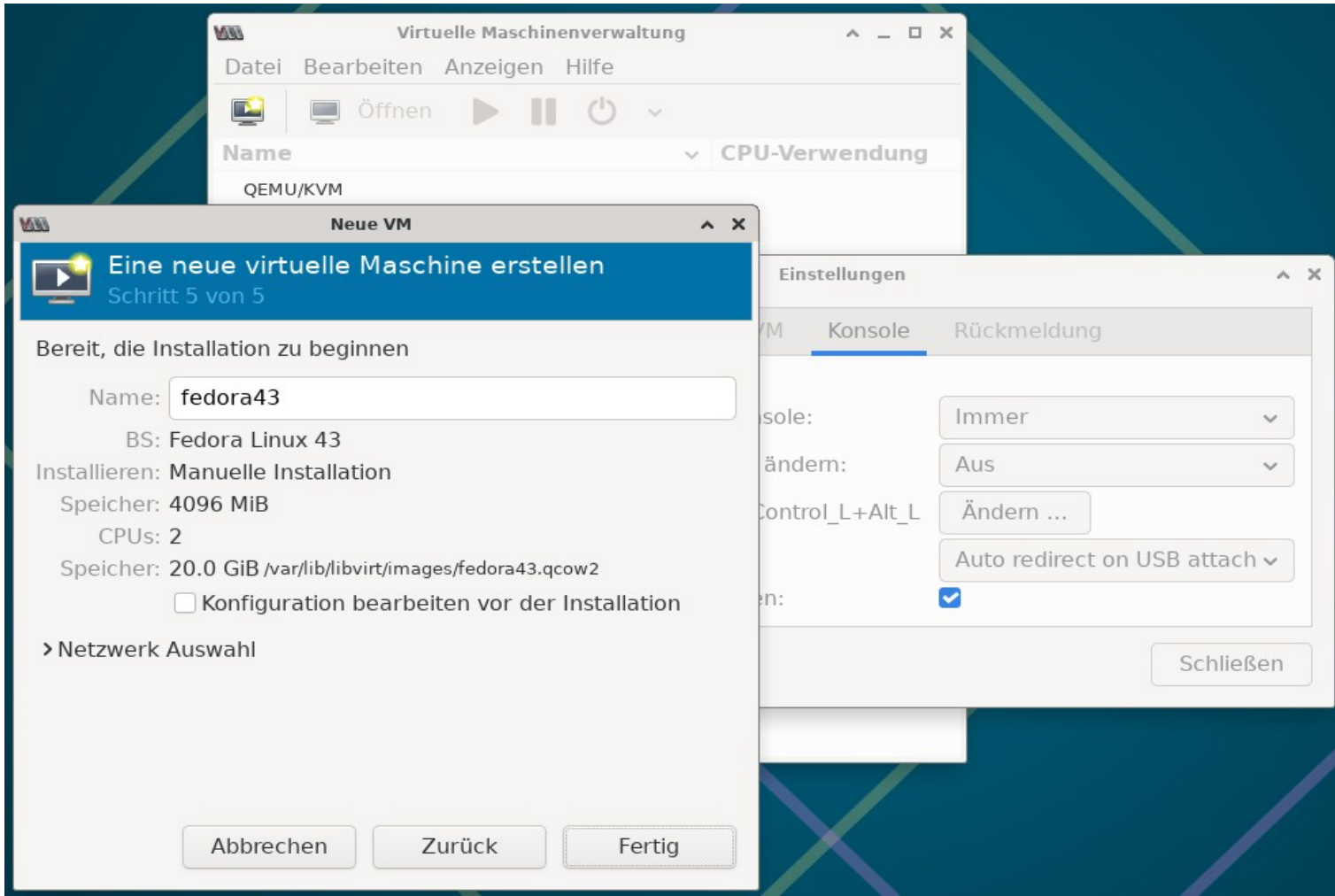
- `virsh --help`
- z.B. Starten und Stoppen von Maschinen:

```
virsh start vmname
```

```
virsh shutdown vmname
```

```
virsh destroy vmname
```

# virt-manager



Danke für die Aufmerksamkeit!

—

Fragen?

